

Язык Rust

(Базовые знания)

(Часть 1)

Занятие 3



Кафедра блокчейна

План занятия

Лекция (теория)

Почему Rust и где он
используется

Синтаксис базового уровня:
переменные, типы, условия,
циклы

Функции и ассоциированные
функции

Структуры, перечисления,
match

Семинар (практика)

Владение (ownership),
borrowing

Память: stack vs heap
move / copy / clone

& и &mut, правила
заимствования

Коллекции: Vec, String,
HashMap

Введение в Rust

Ключевые идеи

Безопасность памяти без GC: Rust ловит большинство ошибок с памятью ещё при компиляции – программа либо не соберётся, либо будет безопасной (без “висячих ссылок” и двойного освобождения).

Zero-cost abstractions: можно писать удобный, “высокоуровневый” код, но по скорости он обычно как ручной C/C++ – без скрытых тормозов в рантайме.

Понятное владение данными: всегда ясно, кто хранит данные, кто имеет право их менять и когда данные освобождаются.

Сильная экосистема: много готовых библиотек и инструментов для системного программирования и задач из блокчейна (сеть, криптография, производительность).

Минимальная программа

```
fn main() {  
    println!("Hello, Rust!");  
}
```

Как запускать Rust код

Варианты

Rust Playground:
<https://play.rust-lang.org/>
Локально: rustup +
cargo
IDE: VS Code +
rust-analyzer

Локальная установка (пример)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustc --version
cargo --version
cargo new hello
cd hello && cargo run
```

Совет

В Playground удобно делиться ссылкой на код и видеть ошибки компилятора
В проектах используем Cargo (зависимости, сборка, тесты)

Память: Stack vs Heap

Интуиция

Stack: быстрый, фиксированные размеры, значения живут по scope

Heap: динамический размер (String, Vec), нужна аллокация/деаллокация

String/Vec – «хедер» на stack + данные на heap

Что лежит где (упрощённо)

```
let n: i32 = 5;           // stack
let s: String = "hi".into(); // header on stack,
let v: Vec<i32> = vec![1,2,3]; // bytes on heap
```

STACK
(хедеры)

HEAP
(данные)

Переменные и мутабельность

Правила

- По умолчанию переменные неизменяемые (immutable)
- `mut` — разрешает изменение
- Shadowing: можно переобъявлять имя с новым типом/значением

Примеры

```
fn main() {  
    let x = 10;  
    // x = 11; // error  
  
    let mut y = 10;  
    y += 1;  
  
    let x = x + 1; // shadowing  
    let x = "теперь строка";  
}
```

Типы данных (скалярные)

Основные типы

Целые: i8/i16/i32/i64/i128,
isize; u8/u16/u32/u64/u128,
usize
Float: f32, f64
bool, char
Строки: &str, String
Кортежи (tuple) и массивы
(array)

Примеры объявлений

```
let a: i32 = -42;  
let b: u64 = 1_000_000;  
let pi: f64 = 3.1415;  
let ok: bool = true;  
let ch: char = 'Ж';  
  
let s_slice: &str = "hi";  
// строковый срез  
let s_owned: String =  
String::from("hi"); // владение строкой  
(heap)  
  
let t: (i32, &str) = (7, "hi");  
let arr: [u8; 3] = [1, 2, 3];
```

Условия (if / else)

Особенности

if — выражение: может возвращать значение
Условия всегда bool (нет 'truthy')
Фигурные скобки обязательны

if как выражение

```
let n = 10;  
let kind = if n % 2 == 0 { "even" } else {  
  "odd" };  
  
if n > 5 {  
  println!("big");  
} else {  
  println!("small");  
}
```


Циклы (loop / while / for)

Примеры

Когда что использовать

loop — бесконечный, можно break с возвращаемым значением

while — пока условие истинно

for — итерация по диапазону или коллекции

```
let mut i = 0;
let v = loop {
    i += 1;
    if i == 3 { break i * 10; }
};

while i < 5 { i += 1; }

for x in 0..3 {
    println!("{}", x);
}
```

Функции

Ключевые моменты

Параметры и возвращаемое значение
типизированы

Последнее выражение без ; — это возвращаемое
значение

return можно писать явно, но чаще используют
выражение

Если ничего не возвращаем — тип ()

Пример

```
fn add(a: i32, b: i32) -> i32 {  
    a + b  
}
```

```
fn main() {  
    let x = add(2, 3);  
    println!("{}", x);  
}
```

// Последнее выражение без ';' — это
return

Ownership: кто владеет значением?

3 правила

У каждого значения есть владелец (owner)

В момент времени — один владелец

Когда владелец выходит из scope — значение освобождается

move по умолчанию (heap данные)

```
let s1 = String::from("hello");  
let s2 = s1; // move  
// println!("{}", s1); // error  
println!("{}", s2);
```

s1 (было)
владеет heap

s2 (теперь)
владеет heap

move / copy / clone

Разница

move — перенос владения (для heap-типов)

Copy — побитовое копирование (i32, u64, bool, char, ссылки)

clone() — явное глубокое копирование (может быть дорогим)

Пример

```
let a: i32 = 7;  
let b = a;           // Copy  
println!("{}", a, b);
```

```
let s1 = String::from("hi");  
let s2 = s1.clone(); // deep copy  
println!("{}", s2);
```

Функции забирают ownership

Пример

Ключевые моменты

Передали в функцию аргумент? Вы передали ownership в эту функцию!

```
fn main() {  
    let s = String::from("hello");  
    takes_ownership(s);  
    println!("{some_string}"); // Ошибка!  
}  
  
fn takes_ownership(some_string: String) {  
    println!("{some_string}");  
}
```

Функции возвращают ownership

Пример

Ключевые моменты

Когда функция возвращает значение - она передает ownership

```
fn main() {  
    let s1 = gives_ownership();  
    println!("{s1}")  
}  
  
fn gives_ownership() -> String {  
    let some_string = String::from("hello");  
    some_string  
}
```

Структуры (struct)

Что важно помнить

struct – «объект» данных (как класс без наследования)

impl блок – методы и ассоциированные функции

Методы берут self / &self / &mut self

Задание - сделать Структуру “человек”. Есть поле “имя”. У него есть Функция New и метод “выведи свое имя в консоль”

```
String::from(“имя”)
```

Пример struct + метод

```
struct Point { x: i32, y: i32 }  
impl Point {  
    fn new(x: i32, y: i32) -> Self {  
        Self { x, y }  
    }  
    fn move_by(&mut self, dx: i32, dy: i32) {  
        self.x += dx; self.y += dy;  
    }  
}  
  
let mut p = Point::new(1, 2);  
p.move_by(3, 4);
```

Ассоциированные функции

Что это такое

Функции в `impl` без `self` (не методы)

Вызываются как `Type::new()`, `Type::from(...)`

Часто используются как «конструкторы» и фабрики

`Self` — псевдоним типа, удобно возвращать `Self`

Пример

```
struct User { id: u64 }

impl User {
    fn new(id: u64) -> Self {
        Self { id }
    }

    fn from_str(s: &str) -> Self {
        let id: u64 =
s.parse().unwrap_or(0);
        Self { id }
    }
}

let u = User::new(42);
let u2 = User::from_str("100");
```


Перечисления (enum)

Зачем нужны enum

Один тип — несколько вариантов данных

Часто используется с `match` для безопасного ветвления

`Option<T>` и `Result<T, E>` — базовые типы `std`

Пример enum

```
enum Msg {  
    Ping, // no type  
    Text(String), // String type  
    Move { x: i32, y: i32 }, // Struct  
type  
}  
  
let m = Msg::Text("hi");
```

Встроенные enum

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

```
enum Option<T> {  
    None, // no type  
    Some(T), // type T  
}
```

```
let some_number = Some(5);  
let some_char = Some('e');
```

```
let absent_number: Option<i32> =  
Option<i32>::None;
```

Match: паттерны и исчерпываемость

Пример match

Почему match мощнее, чем switch:

- 1) Нельзя “забыть” вариант: компилятор требует обработать все случаи (или явно указать `_`).
- 2) Умеет “разбирать” данные: можно сразу вытаскивать значения из `enum`, кортежей и структур прямо в ветке.
- 3) Есть условия в ветках: можно добавлять `if`-проверки (`guards`), чтобы уточнять, когда срабатывает конкретный вариант.

```
match m {  
  Msg::Ping => println!("pong"),  
  Msg::Text(s) => println!("{}", s),  
  Msg::Move { x, y } if x == 0 =>  
    println!("y={}", y),  
  Msg::Move { x, y } => println!("{}",  
    {}, x, y),  
}
```

Строки: String и &str

- 1) `&str` – это срез строки (заимствование, “ссылка на текст”). Обычно вы просто читаете текст и не владеете им.
- 2) `String` – это владеющая строка (данные лежат в heap). Её можно изменять: добавлять, удалять, расширять.

Длина / размер строки:

`s.len()` возвращает количество байт, а не “количество букв”.

Что такое `into()` и зачем:

`.into()` – это удобное преобразование типа через `trait Into`.

Используют, когда функция хочет, например, `String`, а у вас `&str`:

- `"hi".into()` превращает `&str` в `String`

Пример

```
let mut s = "Привет".into();
s.push_str(" Rust");

// осторожно: границы по байтам
let part: &str = &s[0..6];
println!("{}", part);
println!("{}", "hi".len()); //2
```

Заимствование: & и &mut

Пример

Правила borrowing

Можно создать много
неизменяемых ссылок к одному
heap объекту: &T
Или ровно одну изменяемую: &mut
T
Нельзя & и &mut одновременно в
одном scope

```
let mut s = String::from("hi");
```

```
let r1 = &s;  
let r2 = &s; // ok: две неизменяемые  
ссылки
```

```
println!("{}", r1, r2);
```

```
let r3 = &mut s; // ok: только после  
последнего использования r1/r2  
r3.push('!');
```

Коллекции: Vec<T>

Что помнить

Динамический массив на heap
push/pop, индексирование,
итераторы
Возможна ре-аллокация при росте

Пример

```
let mut v = Vec::new();  
v.push(10);  
v.push(20);  
  
for x in &v {  
    println!("{}", x);  
}  
  
if let Some(last) = v.pop() {  
    println!("popped {}", last);  
}
```

HashMap: хеш-таблица

ОСНОВЫ

Ключ → значение (key/value)
entry API удобно для
инкремента/инициализации
Владение: ключи/значения обычно
перемещаются внутрь map

Пример (подсчёт частоты)

```
use std::collections::HashMap;

let words = ["a", "b", "a"];
let mut m: HashMap<&str, u32> =
    HashMap::new();
for w in words {
    *m.entry(w).or_insert(0) += 1;
}
println!("{:?}", m);
```

Практика: задания на дом

Сделайте в Playground

- 1) Функция: принимает `&String` или `&str` и возвращает длину без копирования
- 2) Функция: принимает `&mut String` и добавляет суффикс
- 3) Парсер: из строки "a b a" собрать `HashMap<&str, u32>`

Подход: сначала компилируем → читаем ошибку → исправляем.